

Spilled Energy in Large Language Models

LLM のハルシネーション検出を「エネルギーのズレ」として読む

発表者：山口 海

論文著者：Adrian R. Minut, Hazem Dewidar, Iacopo Masi

arXiv:2602.18671v4 / ICLR 2026 conference paper

論文: <https://arxiv.org/abs/2602.18671> Code: <https://github.com/OmnAI-Lab/spilled-energy/>

論文の狙い: 追加学習なしで誤答検出

問い	着眼点	結論
LLM の回答が 誤答/ハルシネーション かどうかを、外部検索や追加学習なしで検出できるか？	最終 softmax を Energy-Based Model として解釈し、連続する生成ステップ間でそろはずのエネルギーを比較する。	出力 logit だけから計算する spilled energy が、複数タスク・複数 LLM で誤答検出に有効。

Q/A: ``What is the capital of Italy? Answer:``

Logit

The capital of Italy is Rome	✓
The capital of Italy is Sydney	✗

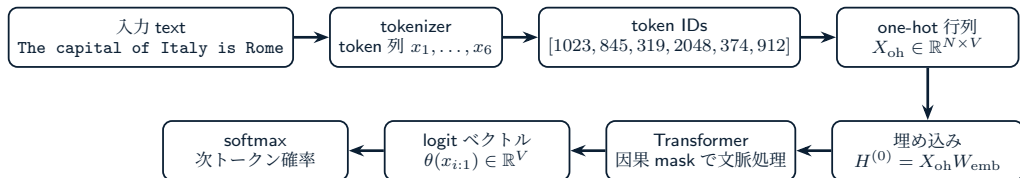
Spilled (Ours)

The capital of Italy is Rome	✓
The capital of Italy is Sydney	✗

核心: 「生成された答えトークンの自信」だけでなく、そのトークンを含む prefix が次の時刻でどれだけ整合的な分布を作るかを見る。

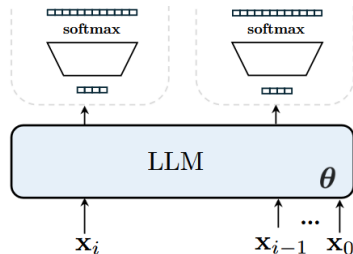
出典: Minut et al., *Spilled Energy in Large Language Models*, Abstract and Sec. 1.

LLM 内部の流れ: text から次トークン確率まで



位置	token x_i	id(x_i)
1	The	1023
2	capital	845
3	of	319
4	Italy	2048
5	is	374
6	Rome	912

LLM 全体を抽象的に $\theta : \mathbb{R}^{N \times V} \rightarrow \mathbb{R}^V$ という関数として表す。 $\theta(x_{i:1})[k]$ は文脈 $x_{i:1}$ の次に語彙 ID k が出る logit である。



ID から埋め込み、logit、softmax へ

1. token ID → one-hot

語彙サイズを V とすると、各 token ID は one-hot 行に変換できる。

$$X_{\text{oh}} = \begin{bmatrix} e_{1023} \\ e_{845} \\ e_{319} \\ e_{2048} \\ e_{374} \\ e_{912} \end{bmatrix} \in \mathbb{R}^{6 \times V}$$

2. one-hot → 埋め込み

$$W_{\text{emb}} \in \mathbb{R}^{V \times d}, \quad H^{(0)} = X_{\text{oh}} W_{\text{emb}} \in \mathbb{R}^{N \times d}.$$

one-hot で、埋め込み行列の該当行を取り出している。

3. Transformer → hidden state

因果 mask により、位置 i の hidden state は prefix だけを見る。

$$H = \text{Transformer}(H^{(0)}), \quad h_i = f_{\theta}(x_{i:1}).$$

4. hidden state → logit → softmax

$$\theta(x_{i:1}) = h_i W_{\text{out}} + b \in \mathbb{R}^V$$
$$p_{\theta}(x_{i+1} = k \mid x_{i:1}) = \frac{\exp(\theta(x_{i:1})[k])}{\sum_{j=1}^V \exp(\theta(x_{i:1})[j])}.$$

例: Rome を生成するステップ

prefix と次トークン

$$\underbrace{(\text{The, capital, of, Italy, is})}_{x_{5:1} \text{ に対応}} \rightarrow \underbrace{\text{Rome}}_{x_6}$$

モデルは prefix $x_{5:1}$ から語彙全体の logit

$$\theta(x_{5:1}) \in \mathbb{R}^V$$

を出す。

Rome の logit

$$\text{id}(\text{Rome}) = 912$$

$$\theta(x_{5:1})[\text{id}(x_6)] = \theta(x_{5:1})[912]$$

が、Rome を選ぶための softmax 前スコア。

softmax 後の確率

$$p_{\theta}(x_6 = \text{Rome} \mid x_{5:1}) = \frac{\exp(\theta(x_{5:1})[912])}{\sum_{k=1}^V \exp(\theta(x_{5:1})[k])}.$$

spilled energy への接続

Rome を検出対象にすると、後で見る量は

$$\Delta E_{\theta}(x_{6:1}) = \text{LSE}(\theta(x_{6:1})) - \theta(x_{5:1})[\text{id}(x_6)].$$

つまり、Rome を出した logit と、Rome を含めた次 step の語彙全体 logsumexp を照合する。

LLM は次の 1 トークンの予想を繰り返す

トークン列

$x_{i:1} = (x_i, x_{i-1}, \dots, x_1)$ と書く。つまり $x_{i:1}$ は「1 番目から i 番目までの文脈」を表す順序付き系列である。

自己回帰モデル

$$p(x_{i:1}) = p_{\theta}(x_1) \prod_{t=2}^i p_{\theta}(x_t \mid x_{t-1:1}) \quad (1)$$

$$= p_{\theta}(x_1) p_{\theta}(x_2 \mid x_1) p_{\theta}(x_3 \mid x_{2:1}) \cdots p_{\theta}(x_t \mid x_{t-1:1}) \cdots p_{\theta}(x_i \mid x_{i-1:1}) \quad (2)$$

ステップ t の条件付き確率は、直前の文脈 $x_{t-1:1}$ から次トークン x_t を予測する。

$$p_{\theta}(x_t \mid x_{t-1:1}) = \frac{p_{\theta}(x_{t:1})}{p_{\theta}(x_{t-1:1})} \quad (3)$$

Step 1: 中間項のキャンセル

(3) を (1) に代入すると、

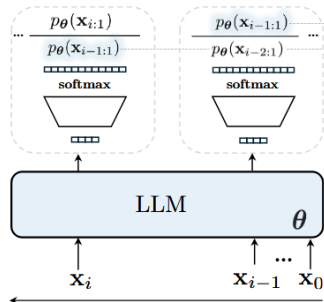
$$\begin{aligned} p(x_{i:1}) &= p_{\theta}(x_1) \prod_{t=2}^i \frac{p_{\theta}(x_{t:1})}{p_{\theta}(x_{t-1:1})} \\ &= p_{\theta}(x_1) p_{\theta}(x_2 | x_1) p_{\theta}(x_3 | x_{2:1}) \cdots p_{\theta}(x_t | x_{t-1:1}) p_{\theta}(x_{t+1} | x_{t:1}) \cdots p_{\theta}(x_i | x_{i-1:1}) \\ &= p_{\theta}(x_1) \frac{p_{\theta}(x_{2:1})}{p_{\theta}(x_1)} \frac{p_{\theta}(x_{3:1})}{p_{\theta}(x_{2:1})} \cdots \underbrace{\frac{p_{\theta}(x_{t:1})}{p_{\theta}(x_{t-1:1})}}_{\text{ステップ } t} \underbrace{\frac{p_{\theta}(x_{t+1:1})}{p_{\theta}(x_{t:1})}}_{\text{ステップ } t+1} \cdots \frac{p_{\theta}(x_{i:1})}{p_{\theta}(x_{i-1:1})} \\ &= p_{\theta}(x_{i:1}). \end{aligned}$$

LLM 実装上

同じ $x_{t:1}$ に関わる量が、

- ▶ 時刻 t : x_t を生成したときの**単一 logit**
- ▶ 時刻 $t+1$: x_t まで含めた文脈から出す**語彙全体の log-sum-exp**

として測られる。



Step 2: LLM の 1-step 予測は softmax 分類器

LLM の 1-step 予測は、語彙上の softmax 分類器である。

$$p_{\theta}(x_i \mid x_{i-1:1}) = \frac{p_{\theta}(x_{i:1})}{p_{\theta}(x_{i-1:1})} = \frac{\exp(\theta(x_{i-1:1})[\text{id}(x_i)])}{\sum_{k=1}^V \exp(\theta(x_{i-1:1})[k])}. \quad (4)$$

分子

実際に生成されたトークン x_i の logit だけを読む。

$$\theta(x_{i-1:1})[\text{id}(x_i)].$$

分母

語彙全体の logit を合計して正規化項を作る。

$$\sum_{k=1}^V \exp(\theta(x_{i-1:1})[k]).$$

EBM では、確率を「低エネルギーほど高確率」と読む。

$$p_{\theta}(x) = \frac{\exp(-E_{\theta}(x))}{Z_{\theta}}.$$

この見方を softmax に適用すると、LLM の 1-step 予測は 2 つのエネルギーに分かれる。

出典: Minut et al., Eq. (3).

Step 3: softmax 分類器を EBM として読む

logit energy

$$E_{\theta}^{\ell}(x_{i:1}) := -\theta(x_{i-1:1})[\text{id}(x_i)].$$

生成されたトークンの logit を負にしたもの。

marginal energy

$$E_{\theta}^m(x_{i-1:1}) := -\log \sum_{k=1}^V \exp(\theta(x_{i-1:1})[k]).$$

次トークン全候補で周辺化した prefix 側のエネルギー。

この新しく定義した 2 つのエネルギーを使うと、LLM の 1-step 予測は

$$\begin{aligned} p_{\theta}(x_i \mid x_{i-1:1}) &= \frac{\exp(\theta(x_{i-1:1})[\text{id}(x_i)])}{\sum_{k=1}^V \exp(\theta(x_{i-1:1})[k])} = \frac{\exp(-E_{\theta}^{\ell}(x_{i:1}))}{\exp(-E_{\theta}^m(x_{i-1:1}))} \\ &= \exp\left\{-E_{\theta}^{\ell}(x_{i:1}) + E_{\theta}^m(x_{i-1:1})\right\}. \end{aligned} \quad (5)$$

となり、EBM の形で解釈できる。

出典: Minut et al., Eq. (4)–(7).

Step 4: 系列全体の負の対数尤度は、エネルギーの差の和になる

(5) を負の対数尤度にとると、

$$-\log p_{\theta}(x_i | x_{i-1:1}) = E_{\theta}^{\ell}(x_{i:1}) - E_{\theta}^m(x_{i-1:1}). \quad (6)$$

となる。ここで系列全体に対して負の対数尤度を取ると、各ステップのエネルギーの差の和になる。

(1) より、

$$\begin{aligned} -\log p_{\theta}(x_{i:1}) &= -\log p_{\theta}(x_1) \prod_{t=2}^i p_{\theta}(x_t | x_{t-1:1}) \\ &= -\log p_{\theta}(x_1) p_{\theta}(x_2 | x_1) p_{\theta}(x_3 | x_{2:1}) \cdots p_{\theta}(x_i | x_{i-1:1}) \\ &= -\log p_{\theta}(x_1) \frac{p_{\theta}(x_{2:1})}{p_{\theta}(x_1)} \cdots \underbrace{\frac{p_{\theta}(x_{t:1})}{p_{\theta}(x_{t-1:1})}}_{\text{ステップ } t} \underbrace{\frac{p_{\theta}(x_{t+1:1})}{p_{\theta}(x_{t:1})}}_{\text{ステップ } t+1} \cdots \frac{p_{\theta}(x_{i:1})}{p_{\theta}(x_{i-1:1})} \\ &= (E_{\theta}^{\ell}(x_{1:1}) - E_{\theta}^m(x_{0:1})) + \cdots + \underbrace{(E_{\theta}^{\ell}(x_{t:1}) - E_{\theta}^m(x_{t-1:1}))}_{\text{ステップ } t} + \underbrace{(E_{\theta}^{\ell}(x_{t+1:1}) - E_{\theta}^m(x_{t:1}))}_{\text{ステップ } t+1} + \cdots \\ &= \sum_{t=1}^i \{E_{\theta}^{\ell}(x_{t:1}) - E_{\theta}^m(x_{t-1:1})\}. \end{aligned} \quad (7)$$

Step 5: そのズレを Spilled Energy と呼ぶ

定義

spilled energy は、理論的には同じ prefix のエネルギーとして一致してほしいが、実際の LLM では異なる時刻かつ異なる softmax 成分から測られる 2 量の差である。

$$\Delta E_{\theta}(x_{i:1}) := -E_{\theta}^m(x_{i:1}) + E_{\theta}^{\ell}(x_{i:1}) = \log \sum_{k=1}^V \exp(\theta(x_{i:1})[k]) - \theta(x_{i-1:1})[\text{id}(x_i)]. \quad (8)$$

timestep i : token logit

$$E_{\theta}^{\ell}(x_{i:1}) = -\theta(x_{i-1:1})[\text{id}(x_i)].$$

timestep $i + 1$: logsumexp

$$-E_{\theta}^m(x_{i:1}) = \log \sum_{k=1}^V \exp(\theta(x_{i:1})[k]).$$

何を比べているか

- ▶ $\theta(x_{i-1:1})[\text{id}(x_i)]$: x_i を生成した時刻の単一 logit.
- ▶ $\log \sum_k \exp\{\theta(x_{i:1})[k]\}$: x_i まで含めた次時刻の語彙全体の正規化項.

具体例: 検出対象を Rome にする

出力を単語単位で簡略化: $x_1 = \text{The}, x_2 = \text{capital}, x_3 = \text{of}, x_4 = \text{Italy}, x_5 = \text{is}, x_6 = \text{Rome}, x_7 = .$

A. 6 番目を出すステップ

入力 prefix: The capital of Italy is

$$\theta(x_{5:1})[\text{id}(x_6)]$$

ここでは、 $x_6 = \text{Rome}$ の logit を読む。

B. 7 番目を予測するステップ

入力 prefix: The capital of Italy is Rome

$$\text{LSE}(\theta(x_{6:1})) = \log \sum_k \exp(\theta(x_{6:1})[k])$$

. の logit 単体ではなく、語彙全体を見る。

$$\Delta E_{\theta}(x_{6:1}) = \text{LSE}(\theta(x_{6:1})) - \theta(x_{5:1})[\text{id}(x_6)].$$

検出対象が Rome なら、Rome を出した logit と、Rome を含めた次 step の語彙全体 logsumexp を照合する。

Q/A: ``What is the capital of Italy? Answer:``

Logit

The capital of Italy is Rome ✓

The capital of Italy is Sydney ✗

Spilled (Ours)

The capital of Italy is Rome ✓

The capital of Italy is Sydney ✗

緑ほど truthful、赤ほど hallucination として可視化。

誤答検出のための比較: 既存手法と Spilled Energy

方法	見るもの	弱点
logit confidence	生成された答えトークンの logit/確率	高確率で誤答を出す「自信過剰」を見逃す。
$p(\text{true})$	「この回答は正しいか？」に対する true の確率	自己評価能力と回答生成能力は一致しない。
probe classifier	hidden state / activation に訓練した分類器	タスク・データセット・層・トークン位置に依存しやすい。
spilled energy	連続生成ステップのエネルギーのズレ	logit アクセスと答え部分の同定が必要。

論文の狙いは「probe を訓練せず、logit confidence より深い信号を取る」こと。

Probe classifier

1. 質問と回答を LLM に通す。
2. 特定層・特定位置の hidden state を取り出す。
3. 小さな分類器を正解/誤答ラベルで訓練する。
4. 新しい回答が hallucination かを予測する。

Spilled energy

1. 質問に対して回答を生成する。
2. 生成時の **logit** 列を保存する。
3. exact answer token に限定する。
4. 追加学習なしで ΔE_θ を計算する。

Probe は表現を学習して読む。Spilled energy は **softmax** の数理構造を直接読む。

出典: Minut et al., Sec. 1.

実用パイプライン: どこにスコアをかけるか

Step	処理	出力
1	LLM に質問を入れて回答を生成。生成時の logit を保存。	回答列
2	回答中の exact answer token を特定。	$[u, w]$
3	各答えトークンで E_{θ}^m と ΔE_{θ} を計算。	token score
4	複数トークンなら pooling。論文では min pooling が良い傾向。	answer score
5	閾値またはランキングで誤答候補を検出。	warning

ポイント: 文章全体ではなく、**意味的に答えを担う短い span** にスコアを絞る。

出典: Minut et al., Sec. 4.2.

Exact answer token が重要

なぜ必要か

- ▶ 句読点・文頭語などは次トークン候補が自然に広い。
- ▶ そのため logsumexp が大きくなり、false positive が出やすい。
- ▶ 真偽信号は「答えの中身」に集中しやすい。

同定方法

- ▶ closed-set: 文字列マッチング。
- ▶ open-ended: 補助 LLM で短い答えを抽出。
- ▶ 抽出失敗時は再試行またはサンプル除外。

方法	Pool	Avg. AuROC
Logit E_{θ}^{ℓ}	Max	56.12
Orgad et al.	Mean	63.67
Marginal E_{θ}^m	Min	67.23
Marginal E_{θ}^m	Max	63.34
Spilled ΔE_{θ}	Min	73.32

exact answer を使うと、Spilled ΔE_{θ} は平均 AuROC で約 24 ポイント改善。

出典: Minut et al., Table 2 and Sec. 5.2.

Pooling: token score を answer score にする

複数 token の答え

答えが複数トークンに分かれるときは、exact answer span $[u, w]$ の各 token にスコアを計算する。

$$\{\Delta E_{\theta}(x_{u:1}), \Delta E_{\theta}(x_{u+1:1}), \dots, \Delta E_{\theta}(x_{w:1})\}.$$

min pooling

$$\min_{i \in [u, w]} \Delta E_{\theta}(x_{i:1})$$

論文で全体的に良い傾向。

max pooling

$$\max_{i \in [u, w]} \Delta E_{\theta}(x_{i:1})$$

局所的な大きな異常を拾う。

mean pooling

$$\frac{1}{w - u + 1} \sum_{i=u}^w \Delta E_{\theta}(x_{i:1})$$

span 全体の平均を見る。

pooling は性能に影響する。論文は method ごとに pooling を ablation し、平均 AuROC で比較している。

実験 1: 合成算術タスクの設計

目的

正解と誤答を制御できる環境で、spilled energy が誤差のある数値回答を分けられるかを確認する。

問題

13 桁以上の整数加算を生成し、正しい解答と誤った解答を用意する。

誤答の作り方

正解にランダムな offset を加える。

- ▶ Easy: $U(10^3, 10^4)$
- ▶ Medium: $U(10^2, 10^3)$
- ▶ Hard: $U(1, 10)$

Hard は数値差が小さく、ぱっと見では plausible。

モデル

Llama-3 8B, Qwen-3 8B, Mistral-7B-Instruct v0.3。

出典: Minut et al., Sec. 5.1.

実験 1 の結果: 誤差が小さくても分離できるか

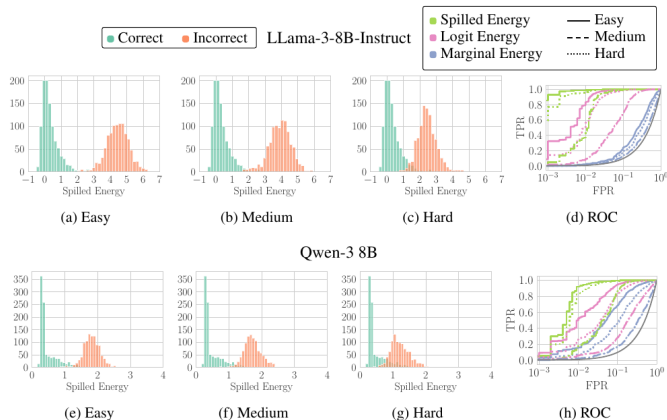
図の読み方

- ▶ ヒストグラム: correct と incorrect のスコア分布。
- ▶ ROC: しきい値を動かした検出性能。
- ▶ 列が右に行くほど難しい。

観察

- ▶ 正解は低い spilled energy に寄る。
- ▶ 誤答は高い spilled energy に寄る。
- ▶ Hard でも logit より強い傾向。

出典: Minut et al., Fig. 3.



実験 2: 実世界 9 ベンチマークの設計

データセット

Math, TriviaQA, HotpotQA, Winogrande, Winobias, Movies, MNLI, IMDB, HotpotQA-WC。

評価したいこと

- ▶ factual QA だけでなく、推論・分類・バイアス系タスクでも信号が出るか。
- ▶ instruction-tuned と base model の両方で機能するか。
- ▶ 追加訓練なしで複数タスクに一般化するか。

出典: Minut et al., Sec. 5.2.

モデル

- ▶ LLaMA 系: base / instruct
- ▶ Mistral 系: base / instruct
- ▶ Gemma 系: 1B / 4B instruct

ラベル

各回答を correct / incorrect に分け、検出スコアの AuROC を計算する。

実世界ベンチマーク: 平均 AuROC の要点

モデル	$p(\text{true})$	Probe	Logit	Best Marginal	Spilled ΔE_θ
LLaMA-Instruct	51.29	64.16	54.62	65.72	73.16
LLaMA base	54.90	62.84	56.89	59.36	68.69
Mistral-Instruct	55.88	65.56	63.44	71.12	77.49
Mistral base	51.36	62.11	49.51	77.19	73.94

読み方

AuROC 50 付近はほぼランダム。 $p(\text{true})$ や logit は平均的に弱く、タスク依存の揺れが大きい。

結論

Spilled energy は多くのモデルで平均性能が高い。ただし Mistral base のように marginal energy が上回る場合もある。

出典: Minut et al., Table 1.

実験 3: Cross-dataset generalization

何を調べるか

Probe classifier をあるデータセットで訓練し、別のデータセットでテストする。

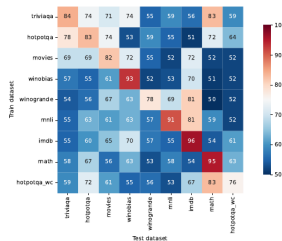
なぜ重要か

実際のユーザー入力はタスクが固定されない。訓練データセットに閉じた検出器は、実運用では使いにくい。

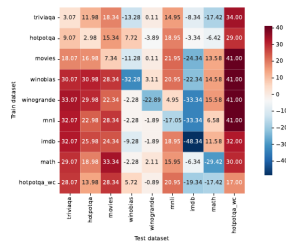
図の見方

右図 (b) の赤い値は、Spilled ΔE_θ が probe より改善している場所。
off-diagonal での改善が重要。

出典: Minut et al., Fig. 4 and Sec. 5.2.



(a) Results by Orgad et al.



(b) Spilled Energy Improvement over Orgad et al.

失敗しやすいケースと限界

false positive

- ▶ 句読点、文頭語、機能語は意味内容が薄い。
- ▶ 次トークン候補が自然に広く、 logsumexp が大きくなりやすい。
- ▶ そのため、答えではない token で高い spilled energy が出ることがある。

対策

exact answer token に限定する。回答全体に一律でかけると、局所的な false positive が混ざる。

実用上の限界

- ▶ white-box な logit アクセスが必要。
- ▶ API によっては語彙全体の logit が取れない。
- ▶ しきい値はモデル・ドメインで校正が必要。
- ▶ 外部事実検証ではないので、真偽の証明にはならない。

位置づけ: RAG/外部検証と併用する**内部不
確実性・異常検出スコア**。

出典: Minut et al., Sec. 5.2, Sec. 6, Appendix D.2.

必要なデータ

- ▶ 生成された token ID 列。
- ▶ 各 step の語彙全体 logit。
- ▶ prompt 長と回答 token 範囲。
- ▶ exact answer span $[u, w]$ 。

各 token で計算

$$\Delta E_{\theta}(x_{i:1}) = \text{LSE}(\theta(x_{i:1})) - \theta(x_{i-1:1})[\text{id}(x_i)].$$

注意点

- ▶ tokenization により、答え文字列と token span がずれる。
- ▶ 生成時 logit と再 forward 時 logit を混同しない。
- ▶ temperature や logit processor の影響を確認する。
- ▶ モデルごとにスコア分布を可視化して校正する。

出力

token-level heatmap、answer-level score、しきい値判定を分けて保存すると解析しやすい。

この論文の貢献を 3 つに整理

数理的貢献

最終 softmax を EBM として読み、chain rule から本来整合するはずの 2 つのエネルギーを取り出した。

方法的貢献

probe classifier や追加訓練なしで、logit 列から token-local な hallucination score を計算した。

実験的貢献

合成算術、9 実世界ベンチマーク、cross-dataset 転移、Gemma 拡張で有効性を示した。

ひと言で: 「モデルの自信」ではなく「生成過程のエネルギー整合性」を見るハルシネーション検出法。

1. LLM の自己回帰分解では、条件付き確率は prefix 確率の比として見られる。
2. LLM はその比を softmax 分類器で実装している。
3. softmax を EBM として読むと、token logit 由来の E_{θ}^{ℓ} と、語彙全体の logsumexp 由来の E_{θ}^m が出る。
4. 同じ prefix $x_{i:1}$ に対して、 $E_{\theta}^{\ell}(x_{i:1})$ と $E_{\theta}^m(x_{i:1})$ は隣接 step で測られる。
5. その不一致が ΔE_{θ} 、つまり spilled energy である。
6. 実験では、正解回答は低い ΔE_{θ} 、誤答回答は高い ΔE_{θ} に寄る傾向がある。

ただし、これは真偽の証明ではない。答え token に絞った内部異常スコアとして使うのが最も自然。

- ▶ Adrian R. Minut, Hazem Dewidar, Iacopo Masi. *Spilled Energy in Large Language Models*. ICLR 2026 conference paper / arXiv:2602.18671v4.
- ▶ arXiv: <https://arxiv.org/abs/2602.18671>
- ▶ Code: <https://github.com/OmnAI-Lab/spilled-energy/>
- ▶ 関連する比較対象: Orgad et al. (2025) probe classifiers, Kadavath et al. $p(\text{true})$, Varshney et al. logit confidence.